DHAKADRIVE-VQA · PLAN FOR THE TEAM

Annotation Platform — Final Plan

How we write, cross-check and analyse VQA questions on dhakavqa.triips.org

14 October 2026 · **Final plan** · Replaces the earlier draft

1 At a glance

WHERE IT RUNS

dhakavqa.triips.org. One Next.js app on **Vercel** does everything, including login. Data lives in **Supabase**; images on **Cloudinary**.

WHO USES IT

One **admin**, who runs everything. **Annotators** write questions on their assigned topics and check each other's.

HOW QUESTIONS ARE CHECKED

Every question is **multiple choice**. **One or two other annotators** check it, always answering **blind** first.

COST

Free tiers to start (Vercel Hobby, Supabase Free, Cloudinary). Upgrade only if a sprint outgrows them.

2 Who does what

Admin (one fixed account)

- Creates, revokes and resets annotator accounts
- Edits the taxonomy and the scene tags
- Runs sprints: topics, targets, verification
- Settles flagged questions; runs the model check
- Sees all statistics; exports the data

Annotator

- Signs in with the credentials the admin issued
- Uploads and tags images
- Writes questions, with their answer, on assigned topics only
- Checks other people's questions during verification

There is no sign-up. Annotators can't change their own password; only the admin can reset it. A new password is shown to the admin once, with a copy button.

3 A sprint, step by step

1 Set up

The admin gives each annotator a whole type or specific subtypes, with a target number of questions.

2 Write

Annotators upload and tag images, then write questions with 2–5 options, marking the correct one.

3 Start verification

Once targets are met, the admin starts verification and writing locks. Each topic gets two checkers who didn't write in it.

4 Check

Checkers work through their queue (next page). Agreement is worked out automatically.

5 Model check

The admin runs a random 10%, covering every subtype, through the models. The app scores the answers automatically.

6 Review & close

The admin settles flagged questions (fix, send back, or drop), then closes the sprint and exports.

4 How answers are checked

Every checker answers **blind first**: they never see anyone else's answer until they've committed their own. What happens next depends on the question's **verifiability**, which comes from its subtype.

V Verifiable

1 checker: is the setter right?

1. The setter gives the answer.
2. The checker answers blind.
3. If the answers differ, the checker now sees the setter's answer and says either *"the setter made a mistake"* (with a reason) or *"I misread it"*.

Accepted if the answers match, or the checker misread.
Flagged if the checker confirms the setter made a mistake.

C Consensus

2 checkers: do people independently agree?

1. The setter gives the answer.
2. Checker A answers blind.
3. Checker B answers blind.
4. Neither ever sees the setter's answer or each other's.

Accepted if all three chose the same option.
Flagged otherwise.

I Inferential

2 checkers: judged one after the other

1. The setter gives the answer.
2. **Second** answers blind, then sees the setter's answer and clicks *Agree* or *Disagree*. Disagreeing needs a reason.
3. **Third** answers blind, then sees both earlier answers and says whether all three agree. If not, they say which answer differs and why.

Accepted if the second agrees and the third says all three agree.
Flagged otherwise.

> For I questions, **agreeing means accepting the answer as reasonable**, not necessarily picking the same option. That's why people decide it, not code. The blind answers are still kept, so we can report agreement both ways in the paper.

On every checking screen

- **"None of these is correct"**: the checker types the right answer. Without this option, a setter who counted wrong can't be caught when the right count isn't one of the options.
- **"Can't tell from the image"**, with a reason. This is a strong sign the question needs rewriting.
- **Shuffled options:** each checker and each model sees the options in a different order, so their position gives nothing away.

Who checks what

- **Two checkers per topic:** neither wrote in it, they're drawn at random but balanced, and the admin can swap them before verification starts.
- **V questions** are split between the two checkers. **C and I questions** go to both.
- **On I questions**, whoever reaches a question first plays "second" and the other plays "third", so nobody waits on a fixed partner.
- **Team size:** at least **three annotators**, and more if several people write in the same topic.

5 Taxonomy

The 8 types and 41 subtypes come from the team's category list. All are stored in the database and **editable by the admin**: types and subtypes can be added, renamed, reordered or hidden, and any subtype's verifiability can be changed. Questions keep the settings they were written with. Every question is **multiple choice**; yes/no questions are simply two-option questions.

V readable from the image C needs a rule or several people I prediction or judgment

★ matters more in South Asian traffic

1 · Perception & fine-grained recognition		
1.1	Object-existence probing (POPE-style)	V
1.2	Fine-grained vehicle classification ★	V
1.3	Attribute recognition (geometry, configuration)	V
1.4	Part-level state (lighting, signalling)	C
1.5	Scene-text recognition / OCR ★	V
2 · Counting & traffic-flow estimation		
2.1	Instance counting (sparse volume)	V
2.2	Class-conditioned counting	C
2.3	Density under occlusion ★	C
2.4	Queue length & level of service (A–F)	C
3 · Road infrastructure parsing		
3.1	Road functional classification	C
3.2	Cross-section element parsing	V
3.3	Junction topology classification	V
3.4	Traffic-control-device recognition	V
3.5	Pavement distress detection ★	C
4 · Spatial grounding & lane topology		
4.1	Ego-lane localisation & region grounding	V
4.2	Monocular depth ordering	V
4.3	Referring-expression comprehension	V
4.4	Amodal completion / occlusion	C
4.5	Figure-ground segregation ★	V
4.6	Pairwise interaction recognition	C

5 · Temporal dynamics & driver behaviour		
5.1	Manoeuvre / action recognition	V
5.2	Kinematic estimation	C
5.3	Gap acceptance	I
5.4	Intent prediction	I
5.5	Trajectory forecasting	I
6 · Rule grounding & right-of-way		
6.1	Visual violation detection	V
6.2	Rule-conditioned violation reasoning	C
6.3	Context-dependent normative reasoning	I
6.4	Right-of-way under posted control	C
6.5	Negotiated right-of-way (uncontrolled) ★	I
7 · Safety-critical reasoning		
7.1	Crash-contributing-factor attribution	I
7.2	Roadworthiness & overload anomaly ★	C
7.3	Vulnerable-road-user exposure ★	C
7.4	Clear-zone & roadside hazards	C
7.5	Sight distance & visibility	C
7.6	Conflict point & crash-type anticipation	I
7.7	Surrogate safety (TTC / PET)	I
7.8	Risk object & countermeasure (CMF)	I
8 · Higher-order reasoning		
8.1	Causal attribution ("why")	C
8.2	Comparative reasoning	C
8.3	Counterfactual & ego-planning	I

14 V · 17 C · 10 I. The less obvious calls: **1.4 is C** because a single frame can catch an indicator mid-blink; **3.1 and 3.5 are C** because they need engineering judgment; **6.1 → 6.3** follow the list's own progression (visible in a frame, then rule-based, then context-dependent). **Category 5** is image-only for now. If its subtypes get flagged much more than the others, that's the signal to add short clips.

6 Scene tags

Each image is tagged at upload, so results can be compared across conditions, for example dashcam vs roadside, or day vs night. The groups and values are editable, and the admin decides which ones are required.

View

Dashcam · Roadside · CCTV / elevated

Time of day

Day · Night · Dawn / dusk

Traffic density

Sparse · Moderate · Dense

Weather & visibility

Clear · Rain · Fog / haze · Dust · Glare

Location

Intersection · Mid-block · Highway · Local street

7 What the admin sees

Overall

- Sprint progress against targets, and checking progress.
- Accepted and flagged rates by type, subtype, verifiability and scene tag.
- Blind agreement between annotators (κ), the figure the proposal says we'll report.
- Model-check results.

Per annotator

- Questions written per subtype, and how many were accepted.
- Checks done, and how often they disagreed (with their reasons).
- Time per question, and activity over time.

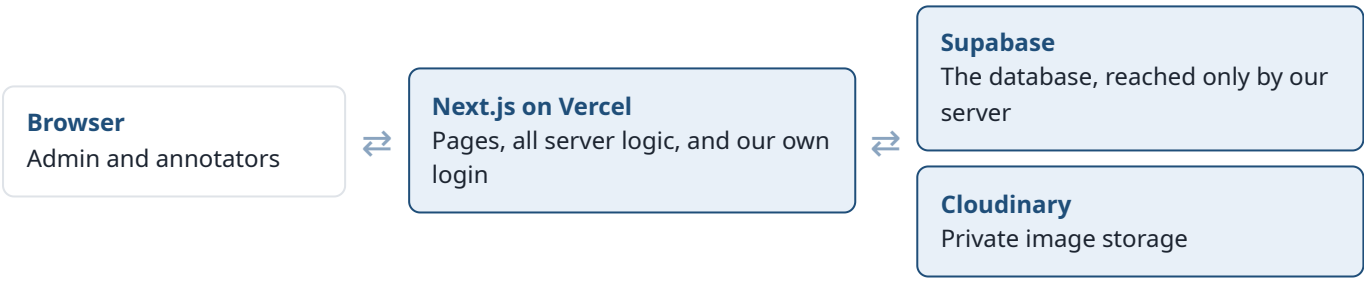
Per question

- The image and its tags; the question, options and the setter's answer.
- Every checker's blind answer and judgment, with reasons and timings.
- The outcome, model answers and edit history.

Export

- JSON and CSV.
- Filtered by sprint, type, subtype, tag or outcome.
- Admin only. Treat exported files as private, since they link to the images.

8 How it's built, and kept safe



- **Our own login, written in Next.js.** The app generates passwords and stores only a scrambled version (a hash). Sign-in locks after repeated failures, and revoking an account signs that person out immediately.
- **Every page and action checks the user's role on the server.** A checker's screen never receives answers they aren't allowed to see yet; those answers aren't merely hidden on the page.
- **The database is reachable only from our server.** Supabase's public API is locked.
- **Images:** they're private on Cloudinary, shown only to signed-in users through signed links.
  - Uploads go straight from the browser to Cloudinary using a short-lived signature, because Vercel caps requests at 4.5 MB.
  - Uploads are limited by file type and size.
- **Secrets and records:** secrets live in Vercel's settings, never in the code. Admin actions are logged, and the database is backed up every night.
- **Hand passwords over privately**, not in group chats.

> **Domain:** triips.org's DNS is managed on **Cloudflare**, so the *dhakavqa* record must be added there, not in cPanel.

9 Build order

1. **Foundation.** The database, login and accounts, with the taxonomy and scene tags loaded.
2. **Annotator side.** Assignments, image upload with tags, and writing questions.
3. **Sprint engine.** Drawing the checkers, the three checking flows, and working out agreement.
4. **Admin side.** Users, the taxonomy and tag editors, sprints, the flagged queue, statistics, the model check and export.
5. **Go live and pilot.** Deploy to dhakavqa.triips.org with backups on, then run a small pilot sprint before the real one.

10 Final thoughts

1. **The design holds together.** Each question gets the checking its type needs: one check for V, two independent checks for C, and a two-step judgment for I. Because every checker commits blind first, the agreement numbers can go straight into the paper.
2. **Multiple choice can hide a setter's mistake.** Setters tend to build the options around their own answer. That's why every checking screen has "None of these" and "Can't tell". Without them, the counting error we're most worried about couldn't be reported.
3. **Set targets with the checking load in mind.** Spread across the 41 subtypes, each question written creates about **1.7 checks**. So 100 questions per person means roughly 170 checks per person.
4. **Pilot first.** A small sprint (about 10 questions each) will show which subtype definitions confuse people, through their flag rates, before we commit to a big sprint. The proposal plans a calibration pilot anyway.
5. **Category 5 from stills is the riskiest part of the taxonomy.** Watch its flag rate; it will tell us whether we need clips.
6. **Free tiers are fine to start.** Supabase Free pauses after a week of no use and has no backups of its own, so the nightly backup matters. Cloudinary's free credits cover storage, bandwidth and resizing together. Revisit both before the first big sprint.

11 What we need to start

- [ ] A **Vercel project** linked to the GitHub repository.
- [ ] **Credentials entered in Vercel's settings:**
  - the Supabase database connection string;
  - the Cloudinary cloud name, API key and secret;
  - the admin username and first password;
  - model API keys, optionally, if the 10% check should run automatically.
- [ ] A **DNS record** for *dhakavqa* in triips.org's Cloudflare account (exact value supplied at deploy time).
- [ ] The names of **at least three annotators** for the pilot sprint.

Based on the DhakaDrive-VQA research proposal (§4.4 diagnostic axes, §4.5 annotation workflow), the team's category list (8 types, 41 subtypes), and the decisions made in review on 12–14 October 2026.